

LAPORAN PRAKTIKUM MINGGU KE-4 KONTROL CERDAS

Minggu Ke-4

Nama : Fajril Nurfianto
NIM : 224308082
Kelas : TKA-7D
Akun GitHub (Tautan) : <https://github.com/fajrilnurfianto28>

1. Judul Percobaan

Reinforcement Learning sebagai Metode Cerdas dalam Autonomous Control

2. Tujuan Percobaan

Pada praktikum ini, mahasiswa diharapkan dapat:

1. Memahami konsep dasar *Reinforcement Learning* (RL) dalam sistem kendali.
2. Mengimplementasikan agen RL menggunakan algoritma *Deep Q-Network* (DQN).
3. Menggunakan OpenAI Gym sebagai simulasi lingkungan untuk pelatihan RL.
4. Melatih dan menguji agen RL untuk mengontrol lingkungan secara otonom.
5. Menggunakan GitHub untuk *version control* dan dokumentasi praktikum.

3. Landasan Teori

Reinforcement Learning (RL) adalah metode pembelajaran di mana *agent* belajar mengambil keputusan melalui interaksi dengan *environment*. *Agent* memperoleh *reward* atau *penalty* sebagai umpan balik dari tindakannya, sehingga dapat mengoptimalkan strategi untuk mencapai tujuan tertentu. Dalam *reinforcement learning*, *agent* belajar secara otomatis menggunakan umpan balik tanpa data berlabel. *Agent* pada *reinforcement learning* berinteraksi dengan *environment* dan mengeksplorasinya secara mandiri. Tujuan utama seorang *agent* dalam *reinforcement learning* adalah meningkatkan kinerja dengan mendapatkan imbalan positif secara maksimal. *Agent* belajar melalui proses *hit and trial* serta berdasarkan pengalaman, sehingga mampu

menyelesaikan tugas dengan cara yang lebih baik (Andreanus & Kurniawan, 2018).

RL tersusun atas beberapa unsur penting sebagai berikut:

- **Agent**

Agent didefinisikan sebagai suatu entitas yang dapat melakukan serangkaian aksi. *Agent* dapat berupa perangkat lunak maupun perangkat keras, bergantung pada jenis *environment* yang dipelajari.

- **Environment**

Environment merupakan lingkungan yang dijadikan sebagai objek pembelajaran *agent*.

- **State**

State adalah kondisi atau keadaan *environment* pada saat diamati oleh *agent*. Nilai *state* akan berubah seiring waktu ketika *agent* melakukan suatu aksi.

- **Reward**

Reward adalah penghargaan yang diterima *agent* selama proses pembelajaran RL. Penghargaan ini dapat berupa hadiah atau hukuman. Hadiah diberikan ketika *agent* memilih aksi yang tepat, sedangkan hukuman diberikan ketika *agent* memilih aksi yang salah.

- **Action**

Action adalah sekumpulan kemungkinan pergerakan atau langkah yang dapat dilakukan oleh *agent* dalam menjelajahi *environment*.

Deep Q-Network (DQN) merupakan pengembangan dari algoritma *Q-Learning* dasar yang menggunakan jaringan saraf tiruan (*deep neural network*) untuk mengaproksimasi nilai-Q. *Deep Q-Learning* termasuk dalam algoritma RL dengan paradigma *model-free*. Secara prinsip, algoritme ini mengasumsikan bahwa *agent* tidak mengetahui model matematis dari *environment* yang dipelajari, seperti peluang transisi pada *state* tertentu maupun *reward* yang akan diperoleh. Algoritma ini secara bertahap membangun “pengetahuan” dari nol melalui proses interaksi dengan *environment*. Properti *deep learning* pada *Q-Learning* berfungsi menggantikan peran *Q-Table* (Febrian, Setiawan, dan Prasetyo t.t.).

4. Analisis dan Diskusi

a. Analisis

Pada praktikum minggu keempat, percobaan yang dilakukan berfokus pada implementasi *Deep Q-Network* (DQN) untuk menyelesaikan permasalahan *MountainCar-v0* menggunakan pendekatan *Reinforcement Learning*.

1) Bagaimana performa agen dalam mengontrol *environment CartPole*?

Performa agen dalam mengontrol *environment CartPole* biasanya diukur dari kemampuannya menjaga tiang tetap seimbang selama mungkin dengan memberikan aksi yang tepat pada kereta. Agen yang baik, terutama yang menggunakan metode reinforcement learning seperti Q-learning atau Actor-Critic, dapat mencapai skor tinggi dengan mempertahankan tiang dalam posisi tegak untuk waktu yang lama, menunjukkan kontrol yang efektif terhadap *environment*. Berikut adalah beberapa

2) Bagaimana perubahan parameter (misal: gamma, epsilon, learning rate) mempengaruhi kinerja agen?

Perubahan parameter seperti gamma, epsilon, dan learning rate sangat mempengaruhi kinerja agen dalam algoritma pembelajaran penguatan (*reinforcement learning*) sebagai berikut:

- Gamma

Gamma yang terlalu kecil atau mendekati 0 bisa membuat agen menjadi "*short-sighted*" dan tidak belajar strategi jangka panjang yang optimal. Gamma mendekati 1 bisa membuat agen terlalu mengandalkan *reward* masa depan yang mungkin tidak pasti, sehingga proses belajar menjadi lebih lambat atau tidak stabil.

- Epsilon (ϵ)

Epsilon yang terlalu tinggi menyebabkan agen terlalu banyak mencoba aksi acak sehingga lambat konvergen. Epsilon yang terlalu

rendah menyebabkan agen cepat terjebak pada solusi lokal dan kurang eksplorasi.

- **Learning Rate (α)**

Learning rate yang terlalu besar bisa menyebabkan nilai Q berosilasi dan tidak konvergen. *Learning Rate* yang terlalu kecil membuat agen lambat beradaptasi terhadap perubahan lingkungan.

3) **Apa tantangan yang muncul selama pelatihan agen RL?**

Meski memiliki potensi yang besar, penerapan *Reinforcement Learning* tidak lepas dari berbagai tantangan antara lain:

- **Eksplorasi dan Eksploitasi**

Menyeimbangkan antara eksplorasi (mencoba aksi baru untuk menemukan strategi yang lebih baik) dan eksploitasi (memanfaatkan pengetahuan yang sudah ada untuk mendapatkan reward maksimal) merupakan tantangan yang sulit.

- **Sparse Reward (Reward Jarang)**

Dalam banyak lingkungan, *reward* yang diterima agen sangat jarang yang menyebabkan lambatnya pembelajaran.

- **Variansi Reward Tinggi**

Reward yang diterima agen bisa sangat bervariasi, terutama dalam lingkungan yang *stochastic* (acak). *Reward* yang tidak konsisten menyulitkan estimasi nilai.

- **Sample Inefficiency**

RL membutuhkan banyak data atau interaksi untuk belajar.

Tantangan ini menjadi fokus riset dan pengembangan metode RL yang lebih efisien, stabil, dan aman untuk berbagai aplikasi nyata.

b. Diskusi

1) **Apa perbedaan utama antara *Reinforcement Learning* dan metode *supervised learning* dalam sistem kendali?**

Perbedaan utama antara *Reinforcement Learning* (RL) dan *Supervised Learning* (SL) dalam konteks sistem kendali adalah sebagai berikut:

a. Reinforcement Learning (RL)

Reinforcement Learning adalah jenis pembelajaran mesin yang mengajarkan agen untuk membuat keputusan melalui interaksi dengan lingkungan dan menerima umpan balik dalam bentuk *reward* (hadiah) atau *punishment* (hukuman). Dalam RL, agen tidak diberi data yang dilabeli seperti dalam *supervised learning*, melainkan harus belajar dari hasil tindakannya sendiri.

b. Supervised Learning (SL)

Supervised learning merupakan metode yang paling populer dalam implementasi algoritma untuk *Machine Learning*. Dimana mesin dilatih untuk memberikan keluaran yang sudah ditetapkan atau diharapkan sebelumnya. Algoritma yang diterapkan pada mesin tersebut ditujukan untuk membuat suatu masukan menjadi keluaran yang diharapkan (Goldstein et al., 2016). Implementasi dari *Supervised learning* digunakan untuk *speech recognition* (pengenalan suara), *credit scoring* (penilaian kredit), *medical imaging* (pencitraan medis), dan *search engines* (mesin pencarian).

Aspek	Reinforcement Learning	Supervised Learning
Data	<i>Reward</i> dan interaksi <i>environment</i>	Data input-output berlabel (kontrol benar)
Feedback	Tidak langsung, berupa <i>reward</i>	Langsung, berupa label output
Tujuan	Maksimalkan <i>reward</i> jangka Panjang	Minimalkan error prediksi output
Interaksi	Aktif, online	Pasif, offline
Kebutuhan model system	Tidak wajib diketahui	Biasanya butuh data control yang lengkap

2) **Bagaimana strategi eksplorasi (*exploration*) dan eksploitasi (*exploitation*) dapat dioptimalkan pada agen RL?**

Strategi eksplorasi (*exploration*) dan eksploitasi (*exploitation*) merupakan aspek kunci dalam *Reinforcement Learning* (RL) yang

menentukan bagaimana agen belajar dan mengambil keputusan. Optimasi keseimbangan antara eksplorasi dan eksploitasi sangat penting agar agen dapat menemukan kebijakan yang optimal. Beberapa pendekatan dan strategi untuk mengoptimalkan eksplorasi-eksploitasi pada agent RL sebagai berikut:

- Epsilon-Greedy dengan Decay

Agen memilih aksi terbaik (eksploitasi) dengan probabilitas $1 - \epsilon$ dan memilih aksi secara acak (eksplorasi) dengan probabilitas ϵ . Nilai ϵ dimulai dari nilai tinggi (misal 1.0) untuk eksplorasi maksimal, kemudian secara bertahap dikurangi (*decay*) menuju nilai kecil (misal 0.01) agar agen lebih fokus pada eksploitasi setelah cukup belajar.

- Softmax / Boltzmann Exploration

Aksi dipilih berdasarkan distribusi probabilitas yang proporsional terhadap nilai estimasi aksi, menggunakan fungsi softmax dari nilai Q .

- Upper Confidence Bound (UCB)

Memilih aksi berdasarkan estimasi *reward* dan ketidakpastian di sekitarnya. Agen memilih tindakan yang mengoptimalkan *trade-off* antara reward yang diketahui dan potensi eksplorasi pada area yang belum dipastikan hasilnya.

- Adaptive Methode

Mengadaptasi tingkat eksplorasi-eksploitasi berdasarkan performa agen.

Optimasi strategi eksplorasi dan eksploitasi secara tepat memungkinkan agen RL mencapai performa terbaik dengan mempercepat proses pembelajaran tanpa kehilangan kesempatan menemukan solusi optimal di ruang aksi yang besar dan dinamis.

3) **Potensi aplikasi lain dari RL dalam sistem kendali nyata apa saja yang dapat diimplementasikan?**

Potensi aplikasi lain dari *Reinforcement Learning* (RL) dalam sistem kendali nyata meliputi berbagai bidang seperti:

- Kendali Robotika

RL dapat digunakan untuk mengendalikan robot dalam tugas-tugas kompleks seperti navigasi, manipulasi objek, dan interaksi dengan lingkungan yang dinamis. Contohnya adalah robot otonom yang belajar berjalan, meraih objek, atau menghindari rintangan.

- Kendali Kendaraan Otonom (*Self-Driving Cars*)

RL dapat mengoptimalkan pengendalian kendaraan otonom, termasuk pengaturan kecepatan, pengereman, dan pengambilan keputusan dalam situasi lalu lintas yang kompleks.

- Otomasi Industri

Otomasi industri menjadi salah satu area besar di mana *Reinforcement Learning* berpotensi membawa perubahan besar. Dalam pabrik atau lini produksi, RL digunakan untuk mengoptimalkan proses manufaktur, seperti pengaturan jadwal produksi, kontrol kualitas, dan pemeliharaan prediktif.

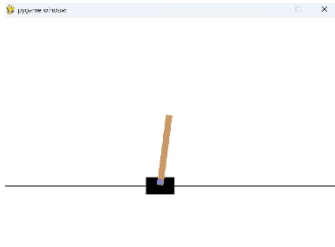
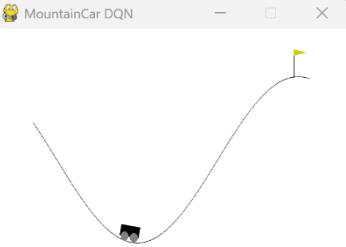
5. Assignment

Dalam praktikum ini, dilakukan pengembangan atau modifikasi agen *Deep Q-Network* (DQN) dengan menambahkan target *network* untuk meningkatkan stabilitas pelatihan. Agen diuji pada *environment* MountainCar-v0 dan CartPole-v1 dengan dukungan visualisasi *real-time* menggunakan Pygame serta grafik dinamis dari matplotlib untuk memantau performa. Penggunaan Keras dan TensorFlow mempermudah pembangunan model, sementara *replay* memory dan strategi epsilon-greedy membantu meningkatkan efisiensi pembelajaran. Meski demikian, visualisasi yang kompleks dan ketergantungan pada banyak pustaka eksternal menyebabkan peningkatan beban komputasi dan waktu pelatihan yang cukup lama, terutama pada *environment* yang lebih rumit.

6. Data dan Output Hasil Pengamatan

Data dan hasil yang diperoleh waktu pengamatan sebagai berikut.

No	Variabel	CartPole-v1	MountainCar-v0
1	Waktu Pelatihan	Agen berhasil mencapai performa yang optimal dengan cepat setelah	Agen memerlukan lebih banyak episode pelatihan karena tingkat kompleksitas lingkungan lebih tinggi serta

		menjalani beberapa ratus episode	tantangan dalam memperoleh reward yang signifikan
2	Jumlah Episode	Sekitar 1000 episode pelatihan, agen mampu mempertahankan keseimbangan tiang secara konsisten <pre> if __name__ == '__main__': # untuk __name__ dan __main__ env = gym.make('CartPole-v1', render_mode=None) # gymnasium state_size = env.observation_space.shape[0] action_size = env.action_space.n agent = DQNAgent(state_size, action_size) episodes = 1000 batch_size = 32 </pre>	Setelah menjalani 1000 episode, agen mulai berhasil mencapai bendera, meskipun proses eksplorasi masih memakan waktu lebih lama sekitar 8 jam dibandingkan dengan pada <i>environment</i> CartPole-v1 <pre> # ---- Main training loop ---- if __name__ == '__main__': env = gym.make('MountainCar-v0', render_mode='rgb_array') state_size = env.observation_space.shape[0] action_size = env.action_space.n agent = DQNAgent(state_size, action_size) episodes = 1000 # cukup 1000 episode dulu batch_size = 64 # batch lebih besar = belajar lebih cepat screen, font = init_pygame() </pre>
3	Skor per Episode (Testing)	Skor rata-rata meningkat signifikan (contoh : 29, 33, 60 hingga > 200) <pre> Episode 5: Score = 29 Episode 6: Score = 33 Episode 7: Score = 60 </pre>	Skor awal mencapai 199 karena maksimal langkah dalam satu periode adalah 200 <pre> Episode: 14/1000, Score: 199, Epsilon: 0.94 Episode: 15/1000, Score: 199, Epsilon: 0.93 Episode: 16/1000, Score: 199, Epsilon: 0.93 Episode: 17/1000, Score: 199, Epsilon: 0.92 Episode: 18/1000, Score: 199, Epsilon: 0.92 Episode: 19/1000, Score: 199, Epsilon: 0.91 Episode: 20/1000, Score: 199, Epsilon: 0.91 </pre>
4	Kinerja Agen	Agen menunjukkan peningkatan yang signifikan, ditandai dengan skor yang terus meningkat secara konsisten	Agen masih menghadapi kesulitan dalam mencapai puncak bukit secara konsisten
5	Kondisi Berhasil	Episode akan berakhir apabila tiang jatuh atau batas maksimum langkah tercapai	Episode berakhir ketika mobil berhasil mencapai tiang bendera yang berada dipuncak bukit
6	Output		

7. Kesimpulan

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat ditarik kesimpulan bahwa hasil pengujian menunjukkan bahwa algoritma *Deep Q-Network* (DQN) dapat digunakan secara efektif untuk mengendalikan berbagai lingkungan simulasi dengan karakteristik yang berbeda. Pada environment CartPole-v1, agen relatif cepat mencapai kinerja optimal berkat pola reward yang konsisten, yang mempermudah proses pembelajaran. Sebaliknya, pada MountainCar-v0, agen memerlukan jumlah episode yang lebih banyak serta strategi eksplorasi yang lebih cermat untuk mencapai tujuan. Temuan ini mengindikasikan bahwa semakin kompleks suatu environment, semakin besar pula tantangan yang harus dihadapi agen dalam proses pembelajaran *Reinforcement Learning* (RL).

8. Saran

1. Untuk menjaga konsistensi selama proses pelatihan, disarankan penerapan teknik seperti target *network* dan *experience replay* dengan kapasitas penyimpanan yang lebih besar. Pendekatan ini akan membuat pembaruan nilai Q menjadi lebih stabil dan mengurangi pengaruh ketergantungan data jangka pendek, sehingga proses pembelajaran menjadi lebih efektif.
2. Selain itu, penting untuk memperluas kajian dengan melakukan perbandingan terhadap algoritma *reinforcement learning alternatif*, seperti Policy Gradient, Actor-Critic, dan Proximal Policy Optimization (PPO). Analisis komparatif ini akan memberikan pemahaman yang lebih mendalam mengenai kelebihan dan keterbatasan masing-masing algoritma, khususnya dalam menghadapi lingkungan dengan tingkat kompleksitas yang lebih tinggi.

9. Daftar Pustaka

- Sutton, R. S. & Barto, A. G., 2018. Reinforcement Learning: An Introduction. 2nd ed. London: The MIT Press.
- Nandy, A. & Biswas, M., 2018. Reinforcement Learning with Open AI, TensorFlow and Keras Using Python. s.l.:Apress.